

Project Juno

An agent that answers questions about customer reviews with exact counts and cited examples.

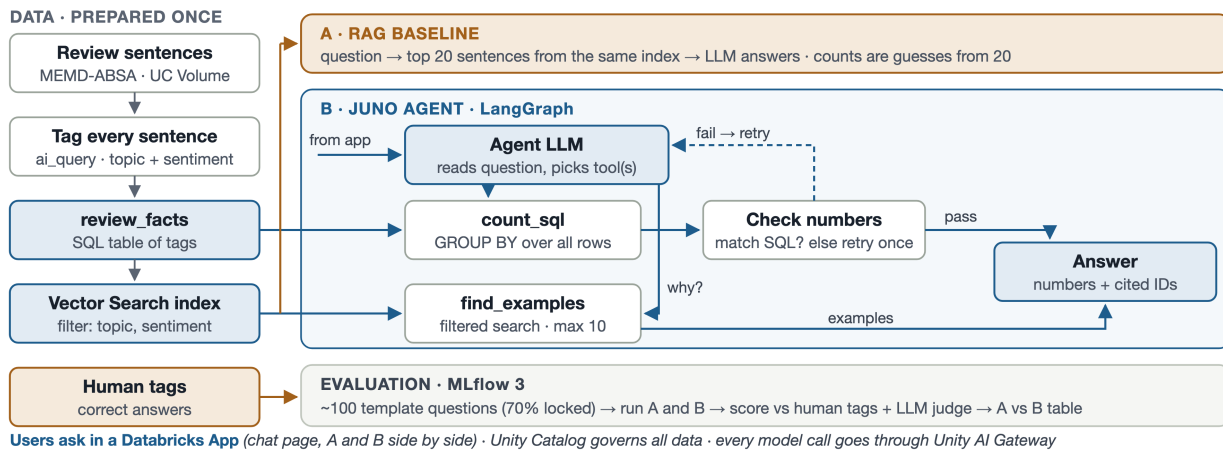
Tamaghna Banerjee · Platform: Azure Databricks

1. Problem statement

Leadership may ask "how many" ("What share of complaints are about service?") and "why" ("Why don't guests like the food?") questions about reviews. The usual RAG chatbot, our baseline, answers from 20 matching sentences, so its counts are guesses. **Juno must get counts, top themes and why-with-examples right.**

Data: MEMD-ABSA Restaurant (Cai et al., 2023): 5,152 review sentences tagged by people with 12 topics and sentiment; these tags are the correct answers. No dates, so no trend questions.

2. Architecture



How it works: An LLM tags every sentence once (topic + sentiment) and saves the tags in a table and a search index. A finds the 20 closest sentences and answers from them. B, the agent, decides per question: count with SQL, or fetch up to 10 examples. It checks its numbers against SQL, retries once, and shows the sentences it used.

3. Evaluation criteria

~100 template questions; correct answers are SQL over the human tags, never Juno's tags. 30% tune, 70% locked test. MLflow scores A and B on the same questions; the LLM judge is checked against 30 hand-graded answers.

Metric	Applies to	Target
Count accuracy	How many	Within 10% on ≥80%; ≥30 points above A
Top-3 match	Top themes	Average overlap ≥0.8 with the correct top 3
Citation accuracy	Why	≥90% of cited sentences match the human tags
Answer quality (LLM judge)	Why	Score ≥4 of 5

Also measured: tag F1 vs human tags, exact SQL number (≥95%), median time (≤15 s), cost per question.

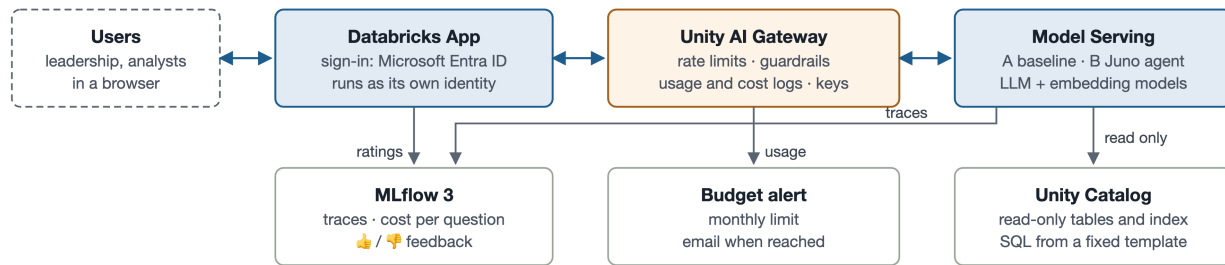
4. Framework

Choice	Why	Instead of
LangGraph	Agent loop as a testable graph; deploys on Databricks	smolagents (hides numbers in code)
Databricks	Unity Catalog, ai_query, Vector Search, SQL, Unity AI Gateway, and Databricks Apps for the user chat page	LLM counting; local stores
MLflow 3	Traces, scorers and LLM judges in one place	A separate eval tool

References: TAG (Biswal 2024, arXiv 2408.14717) · S-RAG (Koshorek 2025, arXiv 2511.08505) · MEMD-ABSA (Cai 2023, arXiv 2306.16956)

Appendix: Access, guardrails and cost

CONTROL PATH · EVERY QUESTION



5. User interface

Users open a Databricks App chat page. Each question goes to A and B, and the answers appear side by side, with Juno's cited sentences under its answer. Users can rate each answer 👍 or 👎; the rating is saved on that answer's MLflow trace.

6. Access and authentication

Question	Control	Where
Who can open the app?	Workspace sign-in (Microsoft Entra ID); access granted only to named users or groups	Databricks Apps
What can the app reach?	Runs as its own identity, allowed only to call the two endpoints	Databricks Apps, Model Serving
What data can the agent read?	Read-only access to the Juno tables and search index; no write rights	Unity Catalog
Where are model keys kept?	Stored once in the gateway, never in code or notebooks	Unity AI Gateway

7. Guardrails

Risk	Guardrail	Where
Off-topic questions	Juno declines anything not about the reviews	Agent prompt and check step
SQL injection or data changes	The LLM only fills a fixed query template (topic, sentiment); SQL runs read-only	count_sql tool, Unity Catalog
Unsafe or personal content	Request and response filtering	Unity AI Gateway
Made-up numbers	Every number checked against SQL; the answer falls back to the SQL result	Check step

8. Cost and monitoring

What	Control	Where
Runaway usage	Rate limits per user and per endpoint; one retry and a timeout per question	Unity AI Gateway, agent
Spend	Monthly budget with an email alert	Databricks budgets
Cost per question	Token counts on every trace, added up per question	MLflow traces
Usage and errors	Request logs and usage tables, reviewed weekly	Unity AI Gateway, system tables